Decision Debate Agent

A Multi-Persona Agentic System for Personal Decision Support

Submitted for the micro1 Agentic Workflows Hackathon | Muhammad Saad Umar | August 2026

1. Overview

Decision Debate Agent is an agentic system that argues both sides of a hard personal decision -- an Optimist and a Skeptic build opposing cases, an Analyst identifies structural trade-offs and missing information, and a Moderator synthesizes all three into one committed, actionable recommendation. A safety pre-check runs before any debate to redirect crisis-adjacent or medical queries to a fixed, non-generated response instead. The system is built with LangGraph, evaluated against a single-prompt baseline across eight real decision scenarios, and exposed through both a command-line evaluation harness and a live web interface.

2. Problem Statement, Motivation, and Target Audience

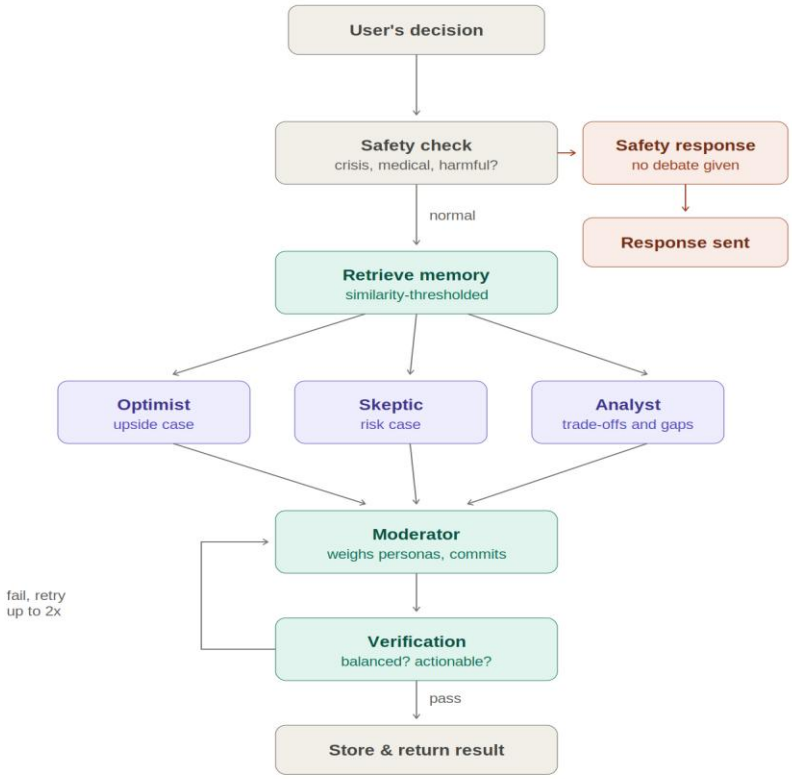
When a person thinks through a hard decision alone, or asks a generic AI chatbot, the output tends to be a long list of considerations that ends in “it depends” without ever committing to a recommendation or naming the specific missing information that would resolve the tension. The target audience is anyone facing a genuinely hard, non-trivial personal decision with real trade-offs -- a job offer, a career pivot, buying versus renting, relocating for a partner -- where reasonable people could land on either side. This project was chosen because it is a real, generalizable, and well-scoped agentic-workflow problem: it does not depend on private data, has a clean way to compare an agent against a simple baseline on the same task, and forcing explicit adversarial reasoning before committing to an answer is a concrete, testable claim about what agentic design adds over a single LLM call.

3. Tech Stack and Rationale

Layer	Tool	Why
Orchestration	LangGraph	Explicit parallel fan-out (personas), fan-in (moderator), conditional retry loop (verification)
LLM	Google Gemini API (free tier)	No-cost access for full build + evaluation; model name kept configurable (free-tier models changed 3x during development)
Memory	ChromaDB + custom TF-IDF embeddings	Vector retrieval with no external model-download dependency; similarity-thresholded to avoid retrieving unrelated past decisions
Backend API	Flask + flask-cors	Thin stateless wrapper exposing the same pipeline used by the CLI and evaluation harness
Frontend	React, TypeScript, Tailwind, shadcn/ui (via Lovable)	Fast iteration on a polished UI without hand-written frontend boilerplate
Evaluation	Custom Python harness + hand-designed rubric	No standard benchmark fit an open-ended decision-support task

4. System Workflow: How a Query Becomes a Recommendation

A user's decision text first passes through a safety-check classifier. If it is flagged as crisis, medical, or clearly harmful, the debate is skipped entirely and a fixed, hardcoded response is returned -- a crisis hotline number, for instance, must never be left to model generation. Otherwise, the system retrieves any genuinely similar past decision from memory (filtered by a minimum cosine-similarity threshold, so an unrelated past query is never mistaken for relevant context), then runs the Optimist, Skeptic, and Analyst personas in parallel against the same input. The Moderator receives all three arguments and produces a single recommendation, explicitly weighing the Optimist-Skeptic tension rather than just listing both sides. A verification step then checks that the draft is genuinely balanced and actionable; if it fails, the Moderator retries with feedback, up to two times. The final result is stored to memory and returned to the user.

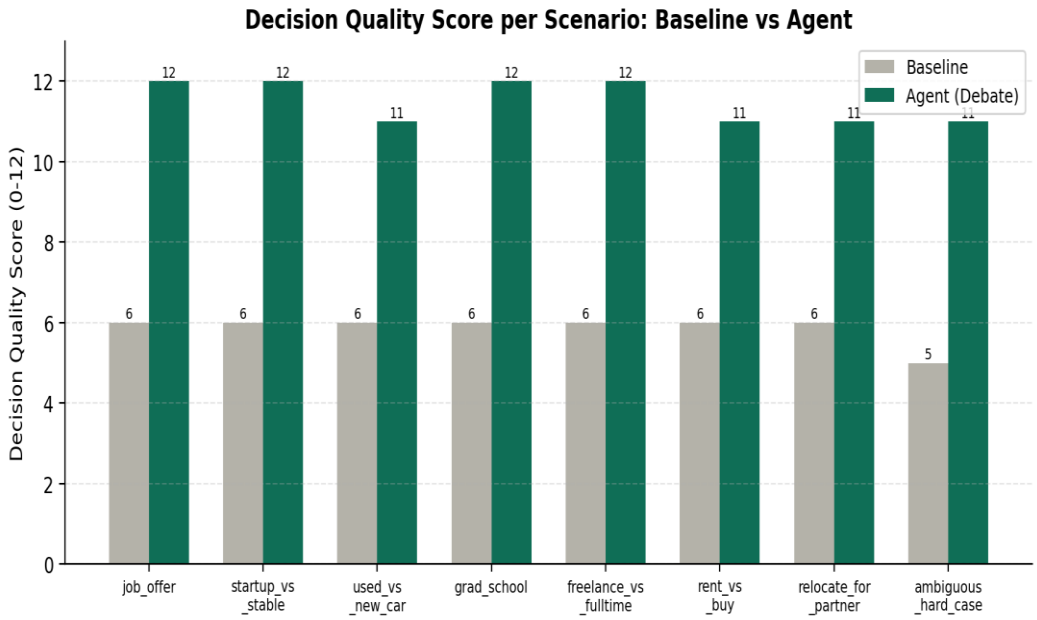


5. Implementation Techniques

- Parallel fan-out / fan-in: three independent LLM calls (Optimist, Skeptic, Analyst) run concurrently as LangGraph nodes and converge on the Moderator -- a control-flow shape a flat, sequential script cannot express as cleanly.
- Conditional retry loop: the verification node can route back to the Moderator with specific feedback, up to two retries, rather than accepting a weak first draft unconditionally.
- Deterministic safety bypass: a classification node routes sensitive queries to a hardcoded response, never an LLM generation, closing a real gap found through deliberate adversarial stress-testing.
- Similarity-thresholded retrieval: a custom local TF-IDF embedding function with a minimum cosine-similarity floor prevents an unrelated stored decision from being treated as relevant context -- a failure mode discovered by manually reading agent output, not by any automated check.
- Resilient API calls: exponential backoff that parses the API's own suggested retry delay, with a separate fast-fail path for daily-quota exhaustion versus per-minute rate limits.

6. Evaluation and Results

Eight decision scenarios (seven typical plus one deliberately ambiguous hard case) were run through both a single-prompt baseline and the full agent, using the same underlying model for both so the comparison isolates the agentic design rather than model quality. Each output was scored 0-3 on four dimensions -- perspective coverage, actionability, specificity, and missing-information awareness -- for a maximum of 12 points per scenario.



Scenario	Baseline	Agent
job_offer	6	12
startup_vs_stable	6	12
used_vs_new_car	6	11
grad_school	6	12
freelance_vs_fulltime	6	12
rent_vs_buy	6	11
relocate_for_partner	6	11
ambiguous_hard_case	5	11
Average	**5.9**	**11.5**

The agent outscored the baseline on every one of the eight scenarios (average 11.5/12 versus 5.9/12). The most consistent gap was actionability: the baseline almost always ended with an unresolved “it depends” framework, while the agent closed with a specific, conditional, numeric recommendation -- a direct result of the Moderator being explicitly forbidden from just listing both sides.

7. Key Finding and Conclusion

The most consequential lesson was not about prompt quality but about scope: deliberate adversarial stress-testing revealed the pipeline would generate a persuasive case for things it never should have touched, such as an “upside case” for discontinuing prescribed medication, and would only handle crisis-adjacent language correctly by accident. Good results on benchmark scenarios do not prove a system is safe -- that required manually reading outputs against sensitive edge-case inputs, not just checking that runs completed without errors. The resulting safety pre-check node closes that gap by design. Overall, this project demonstrates that structured, adversarial multi-agent reasoning -- personas, explicit synthesis, verification, and a hard safety boundary -- measurably improves on single-prompt decision advice, provided the system is tested for what it should refuse to do as rigorously as it is tested for what it should do well.